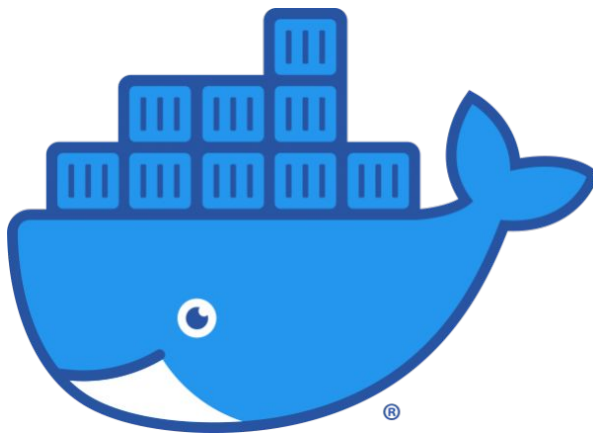


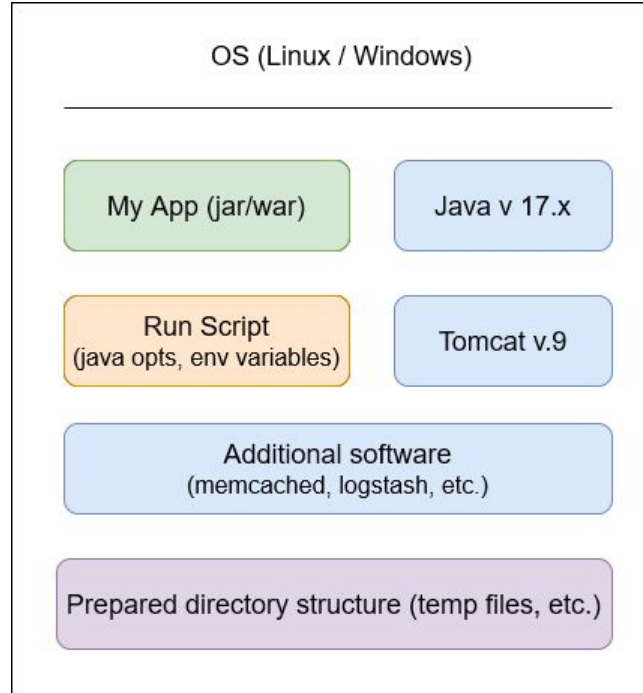
Docker for Developers



Docker was born



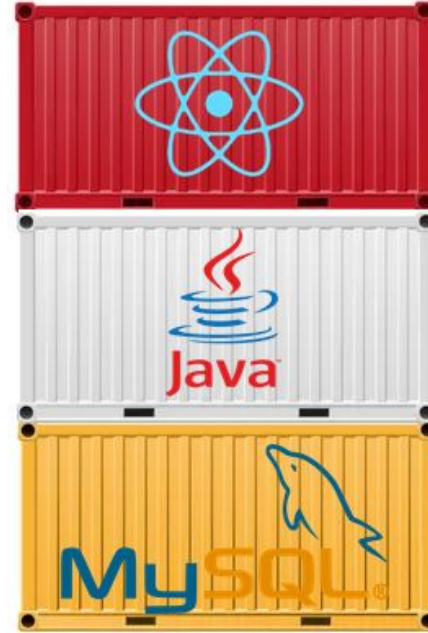
Typical App Deployment



Real life example



Sea freight



App containers



Docker definition

Docker is an open platform for developing, shipping, and running applications in containers.

With Docker, you can **manage your infrastructure** in the same ways you manage your applications.

Docker Components

Client - who connects to **Daemon**

Daemon - docker service itself, which does everything related to docker.

Host - computer on which the **Daemon** service is running.

Container - the smallest element of **Docker**. It is packed, isolated environment in which applications can be launched.

Image - the basis for creating containers. Based on one **image** you can create as many containers as you want.

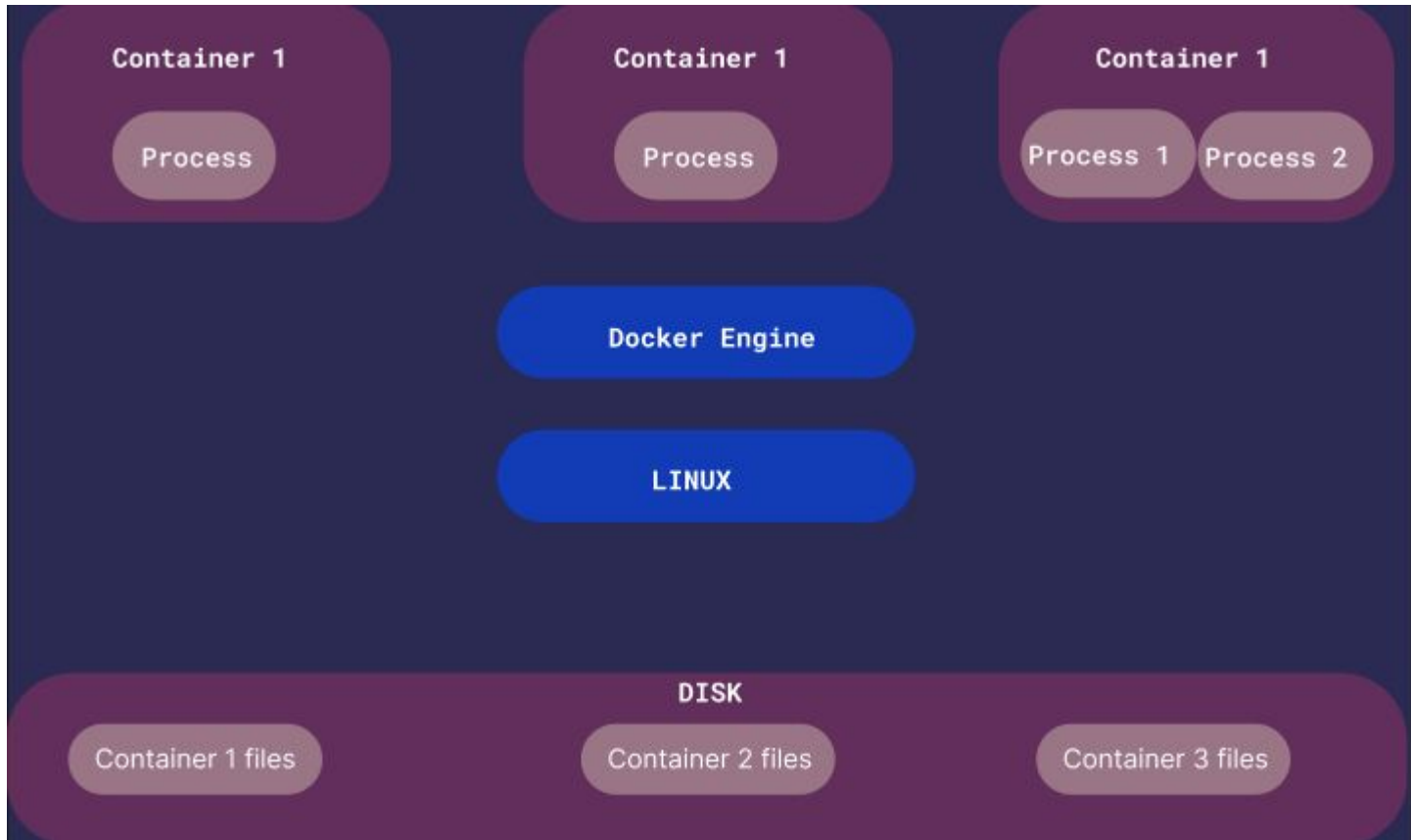
Repository - storage where **images** of different versions are stored

Registry - storage of **REPOSITORIES**

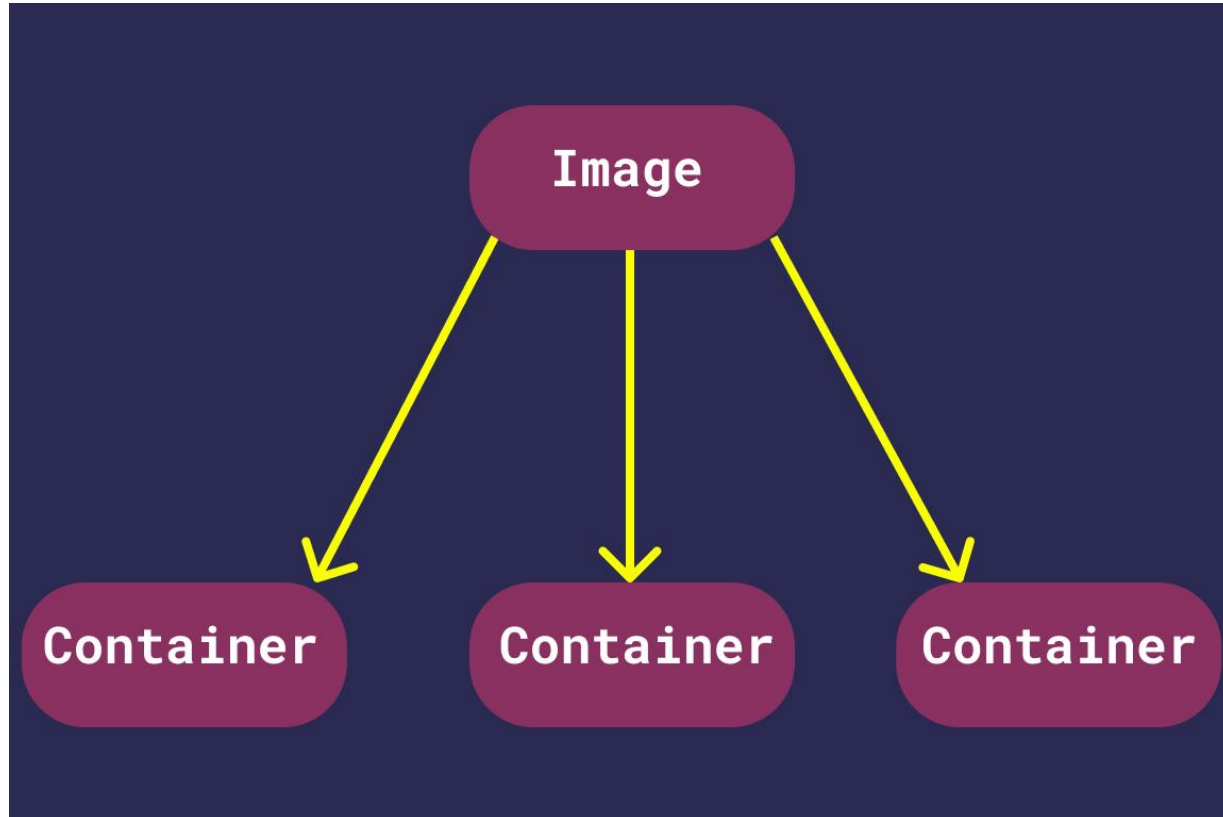
Docker Advantages

- Applications are launched in an `isolated environment`.
- Easy to run on `different servers`.
- All dependencies are installed `inside the container`.
- `Easy to scale` by increasing the number of containers
- Very convenient to `work in development`

Docker Containers



Image



Images Structure

IMAGE

Layer 3

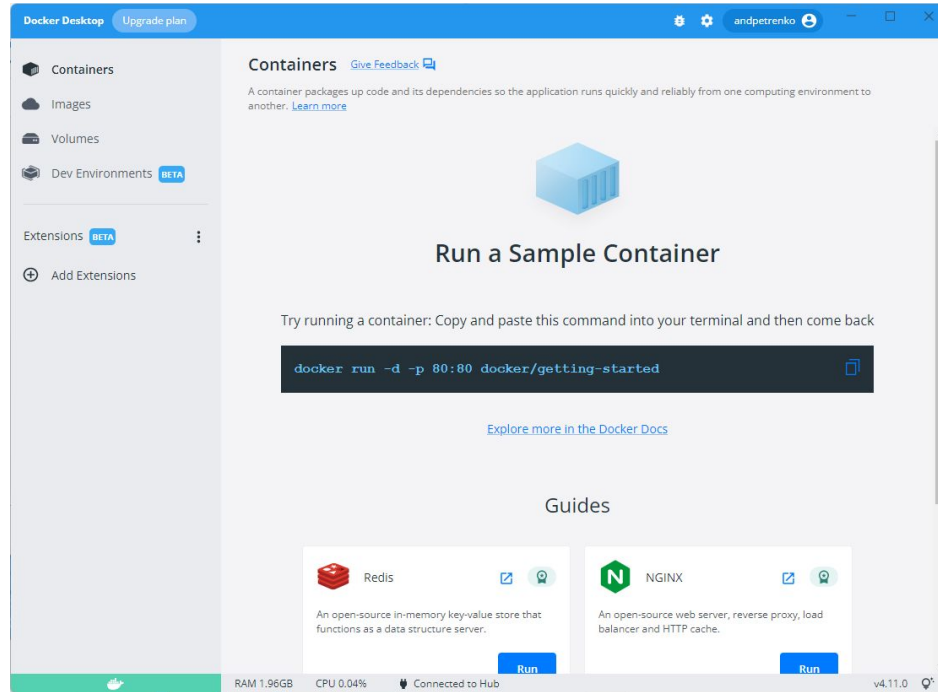
Layer 2

Layer 1

Base layer

- Layers are **read-only**
- Layers can be **transferred** and **deleted**
- Layers are **stored** in repositories

Docker Desktop



<https://www.docker.com/products/docker-desktop>

Basic Docker Commands

download docker image into a local repository

```
docker pull mongo
```

run new container *mymongo* based on image *mongo*

```
docker run --name mymongo mongo
```

start stopped container

```
docker start mymongo
```

execute command inside running container ("-it" means "interactive terminal")

```
docker exec -it mymongo mongosh
```

"-p" allows to map port inside container to our port

```
docker run -d -p 9090:80 imageName
```

see container log

```
docker logs mymongo
```

Explore containers and images

list all running containers

```
docker ps
```

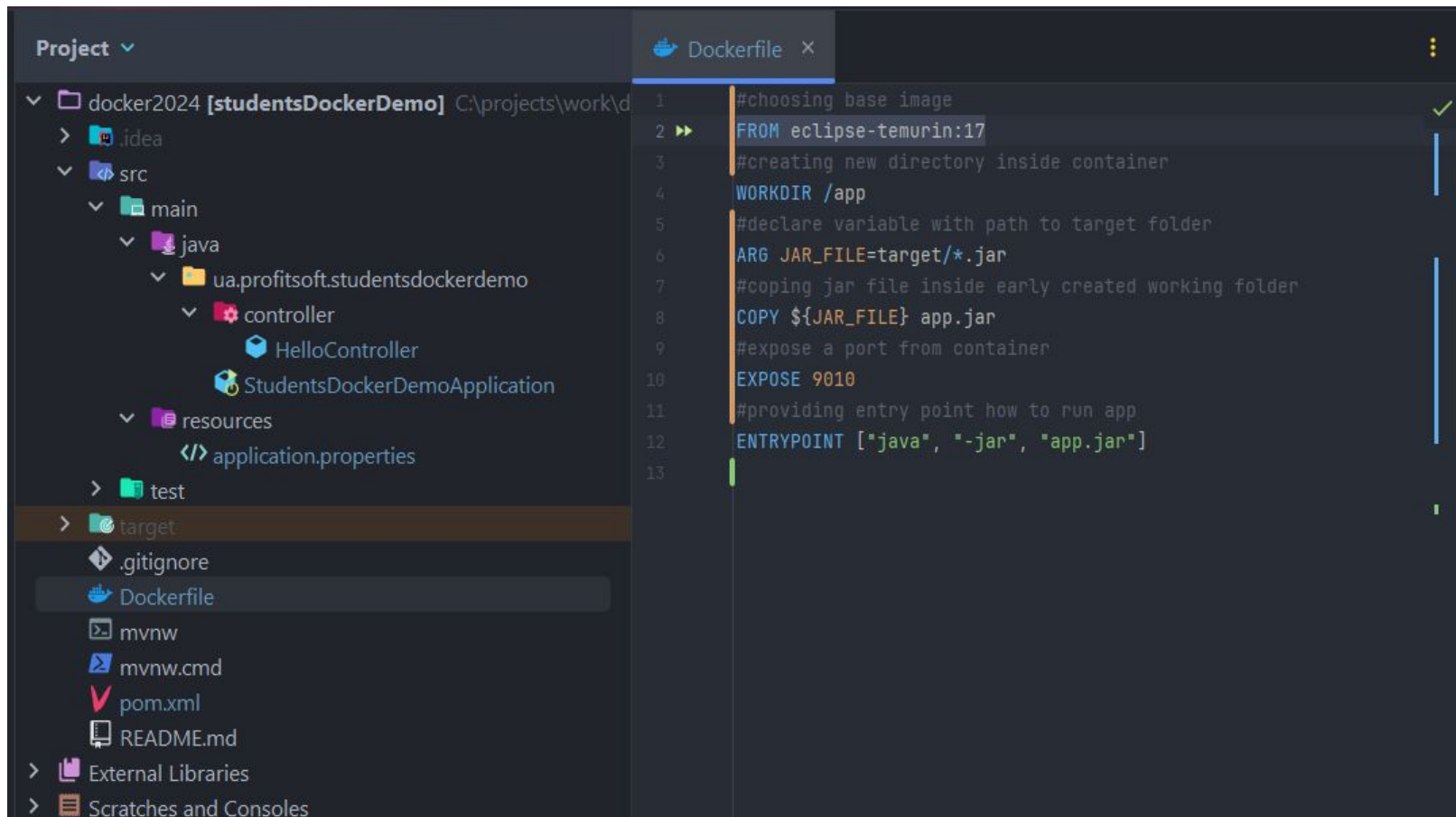
list all running containers (stopped as well)

```
docker ps -a
```

list images in local repository

```
docker image list
```

Containerize simple app



The screenshot displays an IDE interface with a project structure on the left and a Dockerfile editor on the right.

Project Structure:

- Project: docker2024 [studentsDockerDemo] C:\projects\work\docker2024
- src
 - main
 - java
 - ua.profitsoft.studentsdockerdemo
 - controller
 - HelloController
 - StudentsDockerDemoApplication
 - resources
 - application.properties
 - test
 - target
 - .gitignore
 - Dockerfile
 - mvnw
 - mvnw.cmd
 - pom.xml
 - README.md- External Libraries
- Scratches and Consoles

Dockerfile:

```
1 #choosing base image
2 FROM eclipse-temurin:17
3 #creating new directory inside container
4 WORKDIR /app
5 #declare variable with path to target folder
6 ARG JAR_FILE=target/*.jar
7 #copying jar file inside early created working folder
8 COPY ${JAR_FILE} app.jar
9 #expose a port from container
10 EXPOSE 9010
11 #providing entry point how to run app
12 ENTRYPOINT ["java", "-jar", "app.jar"]
13
```

Simple Dockerfile

#choosing base image

FROM eclipse-temurin:17

#creating new directory inside container

WORKDIR /app

#declare variable with path to target folder

ARG JAR_FILE=target/*.jar

#copying jar file inside early created working folder

COPY \${JAR_FILE} app.jar

#expose a port from container

EXPOSE 8080

#providing entry point how to run app

ENTRYPOINT ["java", "-jar", "app.jar"]

Build image and run a container

build your app

```
mvn clean package
```

build docker image

```
docker build --tag simple-app .
```

run docker image

```
docker run -it --name simple-app-container -p 8080:8080 simple-app
```


Multi-stage builds

```
FROM maven as build  
WORKDIR /app  
COPY pom.xml .  
COPY src ./src  
RUN mvn clean package
```

```
FROM eclipse-temurin:17  
#creating new directory inside container  
WORKDIR /app  
#copying jar file from previous stage  
COPY --from=build /app/target/*.jar app.jar  
EXPOSE 8080  
#choosing command to start an app  
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Layering recommendations

- Put `rarely changing` layers first
- Join multiple `RUN` commands `into a single layer`
- `Build artifact` out of your container or use multistage builds

Docker Compose Advantages

Docker Compose is a tool for defining and running multi-container applications.

Advantages:

- **Declarative way** of container creation
- All containers are run **with one command**
- **Automatic creation** of Images based on Dockerfiles
- Automatic **creation of isolated network** for containers interactions

docker-compose.yml

version: "3"

services:

postgres:

image: 'postgres:13.1-alpine'

hostname: db

environment:

- POSTGRES_USER=postgres
- POSTGRES_PASSWORD=master

students-app:

build: .

restart: on-failure

ports:

- "8080:8080"

depends_on:

- postgres

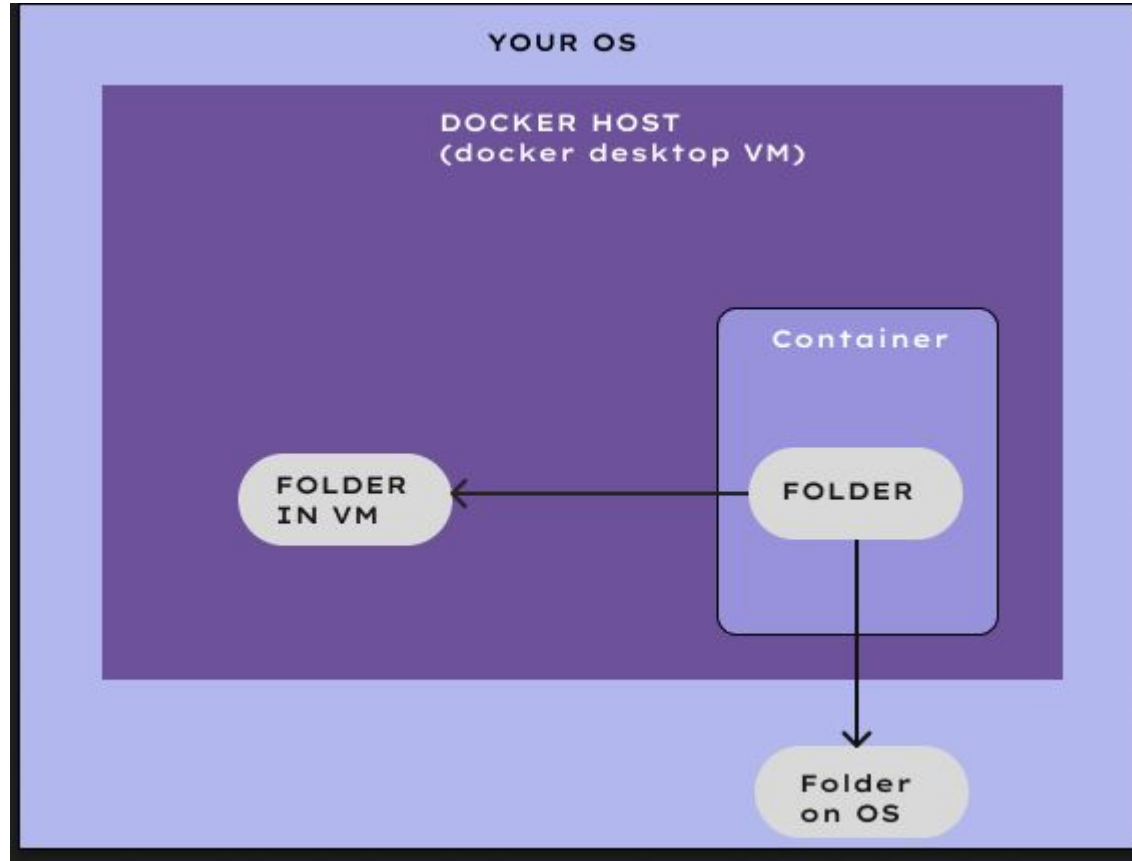
environment:

- SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/postgres
- SPRING_DATASOURCE_USERNAME=postgres
- SPRING_DATASOURCE_PASSWORD=master
- SPRING_JPA_HIBERNATE_DDL_AUTO=update

Run docker-compose

```
# build docker-compose
docker-compose build
# start docker-compose
docker-compose up
# stop docker-compose
docker-compose stop
# remove docker-compose
docker-compose down
```

Volumes



Adding Volumes

version: "3"

services:

postgres:

image: 'postgres:13.1-alpine'

hostname: db

volumes:

- postgres_data:/var/lib/postgresql/data #mapping our volume with volume in container

environment:

- POSTGRES_USER=postgres

- POSTGRES_PASSWORD=master

students-app:

build: .

restart: on-failure

ports:

- "8080:8080"

depends_on:

- postgres

environment:

- SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/postgres

- SPRING_DATASOURCE_USERNAME=postgres

- SPRING_DATASOURCE_PASSWORD=master

- SPRING_JPA_HIBERNATE_DDL_AUTO=update

volumes:

postgres-data:

Network Between Composes

To create a `network` use command "`docker network create networkName`"

To combine two docker-composes we should add the same network for both of them.

#First docker-compose file

```
version: "3"
services:
  hello-app:
    build: .
    networks:
      - shared_network
networks:
  shared_network:
    external: true
```

#Second docker-compose file

```
students-app:
  build: .
  networks:
    - shared_network
networks:
  shared_network:
    external: true
```


Docker Hub



[Explore](#) [Help](#) [Sign up](#) [Sign in](#)

Explore Official Repositories

 nginx official	6.1K STARS	10M+ PULLS	> DETAILS
 redis official	3.8K STARS	10M+ PULLS	> DETAILS
 busybox official	1.0K STARS	10M+ PULLS	> DETAILS
 ubuntu official	6.1K STARS	10M+ PULLS	> DETAILS
 registry official	1.5K STARS	10M+ PULLS	> DETAILS
 alpine official	2.3K STARS	10M+ PULLS	> DETAILS
 mysql official	4.4K STARS	10M+ PULLS	> DETAILS
 mongo official	3.3K STARS	10M+ PULLS	> DETAILS

Docker registry

Docker registry allows to store and share docker images

Run it locally:

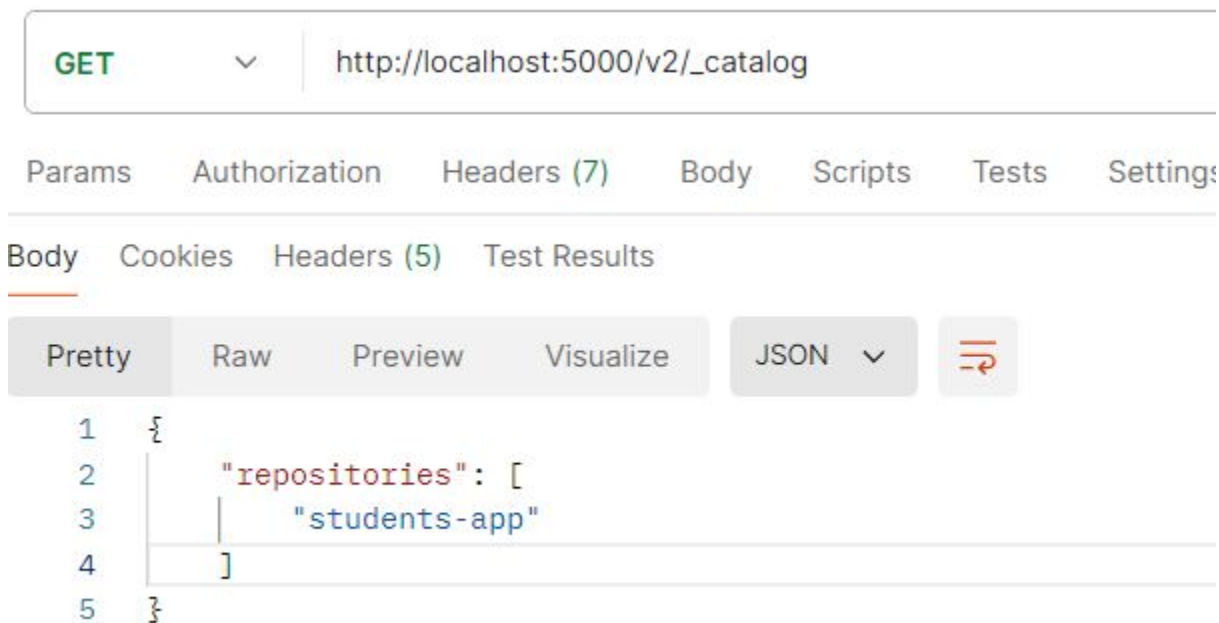
```
docker run -d -p 5000:5000 --name registry registry:2
```

<https://docs.docker.com/registry/>

Push a container to a registry

`docker image tag students-app localhost:5000/students-app`

`docker push localhost:5000/students-app`



The screenshot shows a REST client interface. At the top, a GET request is made to `http://localhost:5000/v2/_catalog`. Below the request bar, tabs for Params, Authorization, Headers (7), Body, Scripts, Tests, and Settings are visible. The 'Body' tab is selected, showing the response in 'Pretty' format. The response is a JSON object with a 'repositories' array containing 'students-app'.

```
1 {
2   "repositories": [
3     "students-app"
4   ]
5 }
```

Reference image from registry

services:

students-app:

image: localhost:5000/students-app

restart: on-failure

ports:

- "8080:8080"

depends_on:

- postgres

...

Keep your laptop clean

**Docker
in Cloud**



**Docker
In My
Machine**



Delete unused containers and images

```
docker image prune [-a]
```